

Task 1:

```
void printPrimes(int k, char *is_prime) {
    if (k <= 100) {
        printf("Primes less than %d: ", k);
        bool first = true;

        for (int j = 2; j < k; j++) {
            if (is_prime[j] == 1) {
                if (!first) printf(", ");
                printf("%d", j);
                first = false;
            }
        }

        printf("\n");
    } else if (k > 100) {
        FILE *fptr;
        char filename[] = "task1_output.txt";

        fptr = fopen(filename, "w");

        if (fptr == NULL) {
            printf("Error: Could not create output file.\n");
            return;
        }

        bool first = true;

        for (int i = 2; i < k; i++) {
            if (is_prime[i] == 1) {
                if (!first) {
                    fprintf(fptr, ", ");
                }

                fprintf(fptr, "%d", i);
                first = false;
            }
        }

        fprintf(fptr, "\n");
        fclose(fptr);

        printf("File '%s' written successfully.\n", filename);
    }
}
```

printPrimes(int k, char *is_prime): Handles the output generation based on the input threshold. If $k \leq 100$, it formats and prints the found prime numbers directly to the standard output (stdout). If $k > 100$, it writes the comma-separated prime numbers to task1_output.txt.

```
bool isPrime(int k) {  
    if (k <= 1) {  
        return false;  
    } else if (k == 2) {  
        return true;  
    } else if (k % 2 == 0) {  
        return false;  
    }  
  
    for (int i = 3; i * i <= k; i += 2) {  
        if (k % i == 0) {  
            return false;  
        }  
    }  
  
    return true;  
}
```

isPrime(int k): Performs initial testing on an k. It uses logic equivalent to trial division up to $\sqrt{k}$ and skips even divisors after checking 2.

```

int main() {
    int k;
    struct timespec start, end;
    double time_taken;

    printf("Enter a number to find prime numbers smaller than it: ");
    scanf("%d", &k);

    if (k <= 2) {
        printf("There are no prime numbers less than %d.\n", k);
        return 1;
    }

    char *is_prime = (char *)calloc(k, sizeof(char)); // using calloc cus want to set everything at 0 first
    // basically cus we want everything to be clear 0 for us to check properly otherwise we get dumb values jic

    if (is_prime == NULL){
        printf("Memory allocation failed\n");
        return 1;
    }

    if (k > 2) {
        is_prime[2] = 1;
    }

    // Get current clock time.
    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 2; i < k; i++) {
        if (isPrime(i)) {
            is_prime[i] = 1;
        }
    }

    // Get the clock current time again
    // Subtract end from start to get the CPU time used.
    clock_gettime(CLOCK_MONOTONIC, &end);
    time_taken = (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) * 1e-9;
    printf("Series prime search up to %d completed in: %lf seconds\n", k, time_taken);

    printPrimes(k, is_prime);
    free(is_prime);
}

```

main(): Coordinates execution by requesting user input *k*, dynamically allocating zeroed memory using “calloc”, isolating pure computation timing using `CLOCK_MONOTONIC`, iterating sequentially over candidates to invoke `isPrime`, and invoking cleanup routines (`free`).

Task 2:

```
void *ThreadFunc(void *pArg) {
    ThreadData *data = (ThreadData *)pArg;

    // this is to parallel compute (HHAHHHAHA) the threads with like evenly distributed digits
    // so like we jump jump per number of threads instead of fixed 1-5, 6-10, 11-15 typa beat
    int starting_point = 3 + (2 * data->thread_id);

    int step_amount = 2 * NUM_THREADS;

    for (int i = starting_point; i < data->k; i += step_amount) {
        if (isPrime(i)) {
            data->is_prime[i] = 1;
        }
    }

    return NULL;
}
```

ThreadFunc(void *pArg): Worker routine executed by each POSIX thread. Uses static partitioning to process a dedicated subset of odd candidate numbers.

For argument we use (ThreadData *)pArg to point to ThreadData and then point to data to use. Unpacks thread configuration parameters (thread_id, k, and is_prime). Calculates a distinct starting point $3 + (2 * \text{thread_id})$ and stride $2 * \text{NUM_THREADS}$ to implement static partitioning. Each thread independently writes results directly into non-overlapping indices of the shared is_prime array, preventing race conditions without needing mutex locks.

Task 3:

```
char *is_prime = (char *)calloc(k, sizeof(char)); // using calloc cus want to set everything at 0
// basically cus we want everything to be clear 0 for us to check properly otherwise we get dun

if (is_prime == NULL){
    printf("Memory allocation failed\n");
    return 1;
}

if (k > 2) {
    is_prime[2] = 1;
}

// Get current clock time.
clock_gettime(CLOCK_MONOTONIC, &start);

#pragma omp parallel for schedule(guided)
for (int i = 3; i < k; i += 2) {
    if (isPrime(i)) {
        is_prime[i] = 1;
    }
}

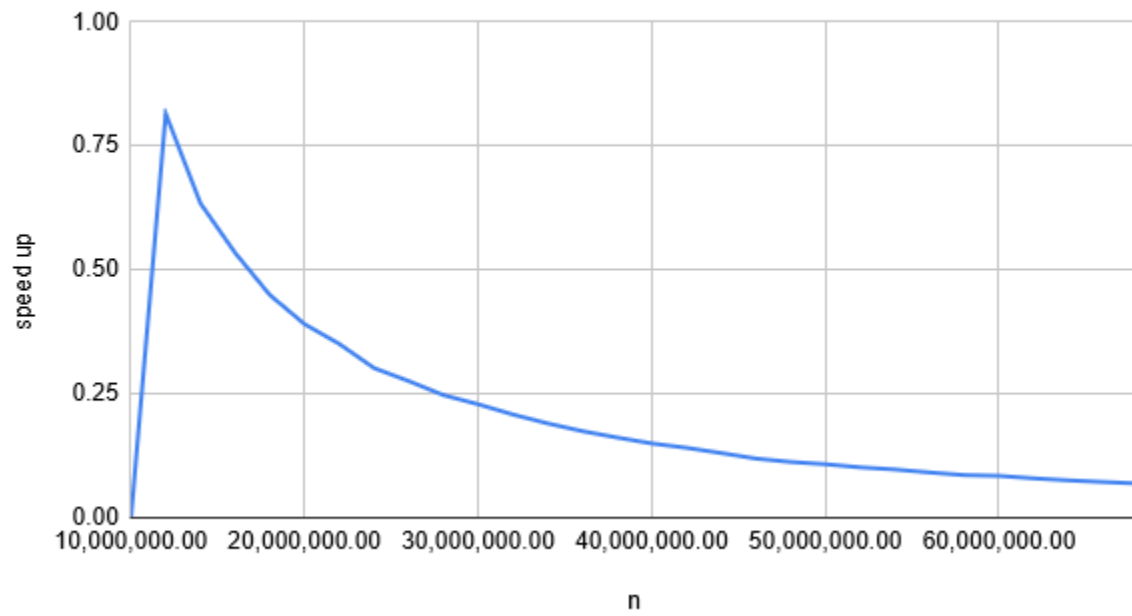
// Get the clock current time again
// Subtract end from start to get the CPU time used.
clock_gettime(CLOCK_MONOTONIC, &end);
time_taken = (end.tv_sec - start.tv_sec) +
             (end.tv_nsec - start.tv_nsec) * 1e-9;
printf("Parallel prime search up to %d completed in: %lf seconds\n", k, time_taken);

printPrimes(k, is_prime);
free(is_prime);
```

#pragma omp parallel for schedule(guided): Directs the compiler to automatically fork a team of worker threads and distribute the iterations of the for loop across them. It uses dynamic guided scheduling to prevent load imbalance caused by variable primality check costs.

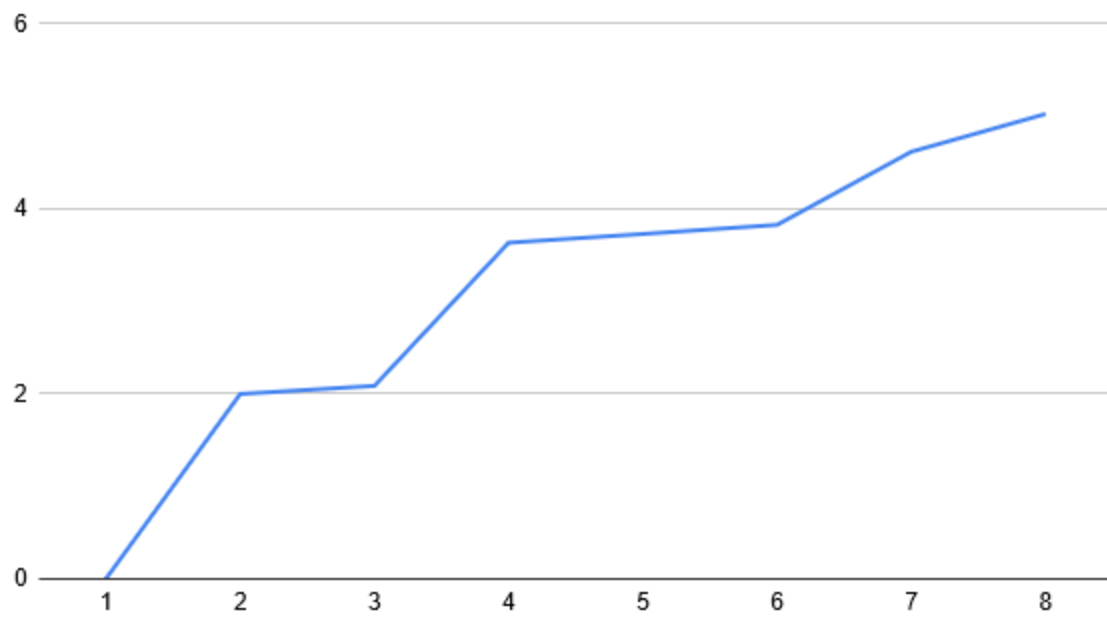
Task 4(b)(1):

speed up vs. n



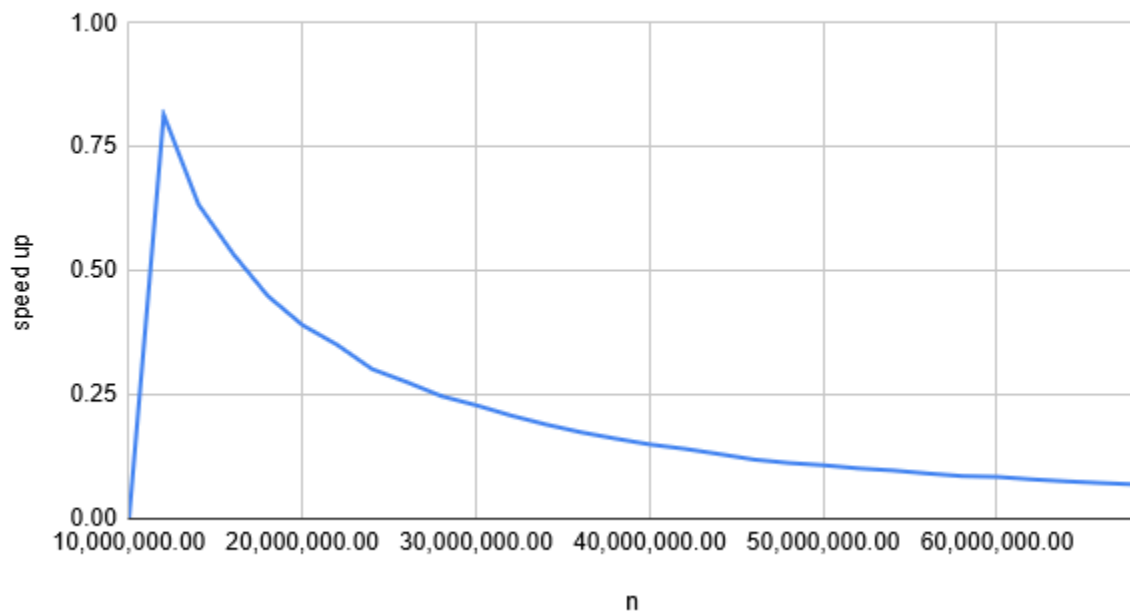
(2)

threads (x) and speed up (y)



(3)

speed up (y) vs. n (x)



(4)

threads(y) and speed up (x)

